Prácticas comparativas de algoritmos de regresión

Guía del proyecto, aclaraciones e instrucciones para el alumnado

IFCD107 — Proyecto de regresión polinómica y casuísticas

30 de octubre de 2026

Índice

1. Presentación y propósito **2**
1.1. Resultados de aprendizaje . . . 2

2. Estructura incluida en el ZIP **2**

3. Datos y casuísticas **2**

4. Protocolo común de entrenamiento **3**
4.1. Por qué el preprocesado va dentro del pipeline . . . 3
4.2. Diferencia entre GridSearchCV y RandomizedSearchCV . . . 3
4.3. Métricas que se deben explicar . . . 4

5. Aclaraciones para completar las plantillas **4**

6. Los 21 métodos y sus puntos de estudio **4**

7. Instrucciones para el ejercicio **5**
7.1. Preparación . . . 5
7.2. Implementación . . . 6
7.3. Ejecución y revisión . . . 6

8. Artefactos que debe entregar el alumnado **6**

9. Rúbrica orientativa **6**

10. Errores frecuentes y cómo evitarlos **7**

11. Referencias de implementación **7**

1 Presentación y propósito

Este proyecto propone completar, ejecutar y comparar 21 algoritmos de regresión sobre seis conjuntos de datos didácticos. Cada algoritmo tiene un entrenador funcional de referencia y una plantilla individual con pseudocódigo comentado que debe completar el alumnado. Las plantillas se encuentran en `plantillas_ejercicio/`; son comentarios de Python y no contienen una solución ejecutable.

El objetivo no consiste en declarar un algoritmo universalmente mejor. Se trata de diseñar un flujo reproducible, evitar fugas de información, justificar la búsqueda de hiperparámetros y explicar cómo cambian los errores entre casuísticas.

1.1 Resultados de aprendizaje

Al completar la práctica, el alumnado podrá:

- preparar variables numéricas y categóricas dentro de un `Pipeline`;
- separar entrenamiento y prueba antes de buscar hiperparámetros;
- comparar `GridSearchCV` y `RandomizedSearchCV` usando los mismos pliegues;
- interpretar RMSE, MAE, sesgo y R^2 sin usar el test para escoger el modelo;
- explicar los hiperparámetros relevantes y los límites de cada familia de modelos;
- conservar los resultados, parámetros, gráficas e informes de forma reproducible.

2 Estructura incluida en el ZIP

- `main.tex`: documento principal del proyecto LaTeX.
- `README.md`: instrucciones breves para compilar, estudiar y usar las plantillas.
- `plantillas_ejercicio/`: 21 archivos `train_* _ejercicio_casuisticas.py`, uno por método, con pseudocódigo detallado en comentarios.

Los entrenadores Python completos se mantienen en el proyecto original. Cada plantilla indica el nombre del entrenador correspondiente para que pueda consultarse como referencia; la carpeta de plantillas del ZIP es el material que se entrega para completar.

3 Datos y casuísticas

Los CSV incluidos originalmente en `proyecto_eda_preprocesado_latex/datos/` son datos simulados. No describen expedientes reales. Las columnas centrales incluyen identificador, distrito, tipo de trámite, superficie solicitada, requerimientos de documentación, mes, predictor `x` y objetivo continuo `y`. El objetivo del ejercicio es predecir `y`.

Archivo	Qué debe analizarse
`01_referencia_limpia.csv`	Referencia sin anomalías intencionadas: flujo base, tipos y distribuciones.
`02_valores_ausentes.csv`	Valores ausentes en variables y objetivo; distinguir imputación de predicciones imposibles cuando falta `y`.
`03_valores_atipicos.csv`	Valores extremos, validación contextual de rangos y sensibilidad de modelos robustos.
`04_categorias_raras.csv`	Categorías poco frecuentes y diferencias de espacios/mayúsculas; normalización y categorías no vistas.

Archivo	Qué debe analizarse
`05_duplicados.csv`	Duplicados exactos y conflictos entre identificador y objetivo; no eliminar filas sin justificarlo.
`06_sesgo_y_heterocedasticidad.csv`	Predictor sesgado y dispersión de `y` creciente con `x`; estudiar residuos y límites de la correlación lineal.

El flujo por lotes lee los seis archivos. También se puede usar la API con un CSV que contenga `y` y al menos una característica. Debe indicarse el significado y las unidades del objetivo antes de interpretar una métrica.

4 Protocolo común de entrenamiento

Cada ejercicio debe seguir el mismo protocolo para que la comparación sea justa.

1. **Inspeccionar y validar.** Leer el CSV, normalizar nombres de columnas, comprobar duplicados, filas vacías, valores ausentes, tipos y presencia del objetivo `y`. Excluir identificadores y columnas que revelen directa o indirectamente el objetivo.
2. **Preparar variables.** Separar características X y objetivo y. Imputar numéricas con estadísticas aprendidas en entrenamiento, codificar categorías y escalar cuando el algoritmo dependa de distancias o magnitudes. Estos pasos pertenecen al `Pipeline`.
3. **Crear la prueba final.** Reservar una partición de test, por ejemplo el 20 %, usando una semilla fija. No ajustar transformaciones, seleccionar características ni elegir hiperparámetros mirando ese conjunto.
4. **Definir las búsquedas.** Crear una cuadrícula acotada y pedagógica para GridSearchCV y distribuciones/rangos razonables para RandomizedSearchCV. Comprobar que los parámetros se aplican al paso correcto del pipeline.
5. **Validación cruzada interna.** Ejecutar ambas búsquedas solo con train, usando los mismos pliegues y el mismo scorer. El preprocesador vuelve a ajustarse dentro de cada pliegue.
6. **Elegir el ganador.** Se usa el RMSE medio de validación cruzada. En scikit-learn, `neg_root_mean_squared_` es negativo; gana el score más alto, es decir, el menos negativo y de menor RMSE.
7. **Evaluar una sola vez.** Evaluar el mejor estimador de cada buscador en el mismo test reservado. Escoger un método por validación, no porque haya salido mejor en test.
8. **Informar y guardar.** Crear gráficas real frente a predicho para ambos métodos y guardar parámetros, métricas, predicciones, informe PDF, modelo y tablas `cv_results_`.

4.1 Por qué el preprocesado va dentro del pipeline

Si se calcula la mediana, la media, la desviación típica o el vocabulario categórico usando todos los datos antes de crear los pliegues, la validación recibe información de observaciones que debería tratar como nuevas. Al colocar la transformación dentro de `Pipeline`, cada pliegue aprende sus transformaciones solamente de su parte de entrenamiento. El test permanece separado hasta la evaluación final.

4.2 Diferencia entre GridSearchCV y RandomizedSearchCV

`GridSearchCV` evalúa todas las combinaciones declaradas en una cuadrícula finita. Es útil para un espacio pequeño y explícito. Su coste crece como el producto del número de opciones de cada parámetro por el número de pliegues.

`RandomizedSearchCV` prueba un presupuesto de candidatos y puede muestrear distribuciones continuas o escalas logarítmicas. No garantiza encontrar el óptimo global; su resultado depende del rango, el número de iteraciones y la semilla. Para una comparación justa, ambos buscadores deben compartir datos de entrenamiento, partición y métrica.

4.3 Métricas que se deben explicar

- **RMSE**: raíz del error cuadrático medio; penaliza más los errores grandes y conserva las unidades de y.
- **MAE**: error absoluto medio; resume el tamaño típico del error con menor sensibilidad a valores extremos que RMSE.
- **Sesgo**: promedio de $y_{\text{real}} - y_{\text{predicho}}$; revela si el modelo tiende a quedarse corto o largo.
- R^2: comparación con una predicción constante basada en la media de entrenamiento. Puede ser negativo en test.

Las métricas no sustituyen el análisis de las gráficas de residuos, la distribución de errores y las condiciones del caso.

5 Aclaraciones para completar las plantillas

- Cada archivo de ejercicio es independiente y corresponde a un único algoritmo. No se debe crear un entrenador que mezcle los 21 métodos en una sola plantilla.
- El archivo de ejercicio contiene pseudocódigo en comentarios. El alumno debe convertir cada paso en Python y completar imports, funciones, validaciones, pipelines, búsquedas, guardado y manejo de errores.
- El entrenador funcional correspondiente sirve para contrastar el flujo una vez que se haya intentado la implementación. No se deben copiar las salidas del entrenador como si fueran una explicación.
- Se busca un resultado didáctico y reproducible, no una garantía de rendimiento predictivo ni una selección automática de variables universal.
- La regresión lineal simple usa exactamente una variable; debe identificarse cuál y justificarse. La regresión lineal múltiple usa varias características válidas.
- Algunas opciones solo aplican a ciertos kernels o estimadores. No deben mezclarse en una cuadrícula configuraciones incompatibles.
- Algoritmos como Gaussian Process, Theil-Sen, CatBoost, XGBoost y MLP pueden consumir más tiempo. Limitar la búsqueda y documentar el coste.
- Para árboles, las categorías deben transformarse de una forma que el estimador admita. CatBoost puede trabajar con categorías nativas si se prepara su entrada de manera coherente; no asumir que todos los estimadores aceptan la misma matriz.

6 Los 21 métodos y sus puntos de estudio

La tabla resume el propósito de cada práctica. Los rangos concretos se justifican en cada plantilla; no son valores universales.

Método	Qué estudiar	Hiperparámetros o decisiones
Lineal simple	Una característica y una recta.	Variable elegida, `fit_intercept`.
Lineal múltiple	Efecto conjunto de predictores.	`fit_intercept`, colinealidad y coeficientes.
Ridge	Penalización L2 estabiliza coeficientes.	`alpha`, escala y colinealidad.
Lasso	Penalización L1 y coeficientes nulos.	`alpha`, convergencia y selección inestable con correlación.
Elastic Net	Combina L1 y L2.	`alpha`, `l1_ratio`.
Bayesiana	Distribución posterior de coeficientes.	Hiperparámetros de precisión e incertidumbre.
Polinomial	Potencias/interacciones para curvas.	Grado y control de crecimiento de características.
Huber	Pérdida menos sensible a residuos grandes.	`epsilon`, `alpha`.
RANSAC	Ajuste con consenso de inliers.	`min_samples`, umbral residual, ensayos y semilla.
Theil-Sen	Medianas de ajustes sobre subconjuntos.	Tamaño de subpoblación, iteraciones y coste.
Cuantílica	Cuantil condicional, no necesariamente la media.	`quantile`, `alpha`, pinball loss.
Árbol de decisión	Reglas locales y riesgo de sobreajuste.	Profundidad, hojas mínimas, criterio.
Random Forest	Promedio de árboles con bootstrap/aleatoriedad.	Número de árboles, profundidad, variables por división.
Gradient Boosting	Ensamble secuencial.	Tasa de aprendizaje, etapas, profundidad y submuestreo.
XGBoost	Boosting con regularización y opciones eficientes.	Profundidad, tasa, subsample y regularización.
LightGBM	Boosting por hojas.	`num_leaves`, profundidad, tasa y hojas mínimas.
CatBoost	Boosting con soporte específico para categorías.	Iteraciones, profundidad, tasa y pérdida.
SVR	Tubo de tolerancia y margen.	Kernel, `C`, `epsilon`, `gamma`.
KNN	Promedio local de vecinos próximos.	Vecinos, métrica, ponderación y escalado.
Gaussian Process	Distribución sobre funciones y predicción incierta.	Kernel, ruido `alpha`, normalización.
MLPRegressor	Red multicapa entrenada por retropropagación.	Capas, activación, regularización y tasa de aprendizaje.

7 Instrucciones para el ejercicio

7.1 Preparación

1. Instalar las dependencias del proyecto original con el entorno Python indicado por el docente.
2. Descomprimir el ZIP en una carpeta de trabajo. Para ejecutar las referencias funcionales, conservar el proyecto original y su carpeta `proyecto_eda_preprocesado_latex/datos/`.
3. Elegir un método y abrir su archivo correspondiente en `plantillas_ejercicio/`.
4. Leer primero la descripción general y el archivo `LEEME_casuisticas.md` del proyecto para entender que los datos son simulados.

7.2 Implementación

Completar en cada práctica, en este orden:

1. carga y validación de una casuística;
2. selección de columnas y preprocesado dentro de un pipeline;
3. partición train/test reproducible;
4. cuadrícula y distribuciones válidas del método;
5. GridSearchCV y RandomizedSearchCV con la misma CV y scorer;
6. comparación de los ganadores mediante validación cruzada;
7. predicción de ambos finalistas en el mismo test;
8. métricas, gráfica real/predicho e interpretación;
9. guardado de modelo, metadatos, tablas de búsqueda y resumen.

7.3 Ejecución y revisión

La estructura del proyecto contiene lanzadores funcionales llamados `train_<algoritmo>_casuisticas.py`; los scripts de práctica terminan en `_ejercicio_casuisticas.py`. Para ejecutar un entrenador de referencia desde la raíz del proyecto se puede usar, por ejemplo:

```
python regression/train_ridge_casuisticas.py
```

Para comparar una entrada mediante API, primero se obtiene un token y luego se envía un CSV a:

```
POST /api/v1/regresion/casuisticas/<algorithm>/csv
```

Campos multipart: `reference`, `file`, `cv`, `n_iter` y `random_state`. La colección Postman del proyecto incluye una solicitud por algoritmo. Los errores de validación deben comunicarse con un mensaje útil y los errores internos deben dejar detalle en el log de ejecución.

8 Artefactos que debe entregar el alumnado

Para el método asignado, entregar:

1. la plantilla completada y comentada con un nivel claro para otra persona;
2. tabla de hiperparámetros y presupuesto de búsqueda;
3. métricas de train/CV/test claramente diferenciadas, sin usar test en el ajuste;
4. predicciones reales y predichas en CSV;
5. gráfica real frente a predicho para GridSearchCV y RandomizedSearchCV;
6. informe con casuística, parámetros ganadores, comparación y limitaciones;
7. reflexión breve sobre generalización, sesgo, outliers, categorías o incertidumbre según el método.

9 Rúbrica orientativa

Criterio	Puntos
Validación de datos y prevención de fuga de información	20
Pipeline/preprocesado adecuado al algoritmo	15
Búsquedas Grid y Randomized válidas, reproducibles y justificadas	20
Selección por validación y evaluación independiente en test	15
Métricas, gráficas y análisis de errores	15
Organización de resultados, documentación y manejo de errores	10
Explicación de límites y conclusiones prudentes	5
Total	100

10 Errores frecuentes y cómo evitarlos

- **Escalar antes de CV**: produce fuga. Incluir escalado dentro del pipeline.
- **Elegir el modelo según test**: convierte el test en validación. Seleccionar por CV y reservar test para la evaluación final.
- **Buscar rangos desproporcionados**: incrementa tiempo sin aportar una pregunta clara. Empezar con rangos pequeños y ampliarlos con justificación.
- **Comparar ejecuciones con particiones distintas**: oculta el efecto del algoritmo. Compartir semilla y holdout.
- **Interpretar importancia o coeficiente como causalidad**: son relaciones del modelo y pueden cambiar con correlación, escala y datos.
- **Ignorar convergencia y coste**: guardar warnings relevantes, número de candidatos, pliegues y duración.
- **Informar solo R^2**: incluir errores en unidades originales y analizar residuos.

11 Referencias de implementación

Las clases y entrenadores del proyecto fuente son la referencia práctica para completar las plantillas. La documentación oficial de scikit-learn describe GridSearchCV y RandomizedSearchCV, pipelines y composición de transformadores, y las métricas de regresión en evaluación de modelos. Para XGBoost, LightGBM y CatBoost, consultar la documentación oficial de cada biblioteca antes de adaptar rangos y opciones.